UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA



Facultad de Ciencias Químicas e Ingeniería

Ingeniero en computación

PROYECTO DE CARRERA

Taller No. 8

PROFESOR:

Jose Isabel Garcia Rocha

ALUMNA: Gutierrez Sanchez Valeria Karey 1282379

GRUPO 581

Taller 8. Arquitectura de hardware y software

1. Propósito y alcance

DashGuard es un sistema de monitoreo y supervisión vehicular formado por un dispositivo con cámara dirigida al conductor, un backend en la nube y una aplicación móvil. Al inicio de cada sesión verifica si quien conduce es una persona autorizada y genera una alerta ante un conductor no reconocido. Registra rutas y excesos de velocidad confirmados por GPS, y permite al supervisor solicitar fotos o videos cortos y consultar el resumen de cada recorrido.

Esta arquitectura define los componentes, las interfaces y los contratos con los que continúa la implementación. La plataforma embebida es la Raspberry Pi Zero 2 W y la nube se compone de Render y Supabase, servicios ya utilizados en la prueba de concepto de envío y consulta de datos GPS. La verificación facial se ejecuta al inicio de la sesión y no de forma simultánea con la grabación de video, de acuerdo con la memoria disponible en la plataforma.

2. Arquitectura general y despliegue

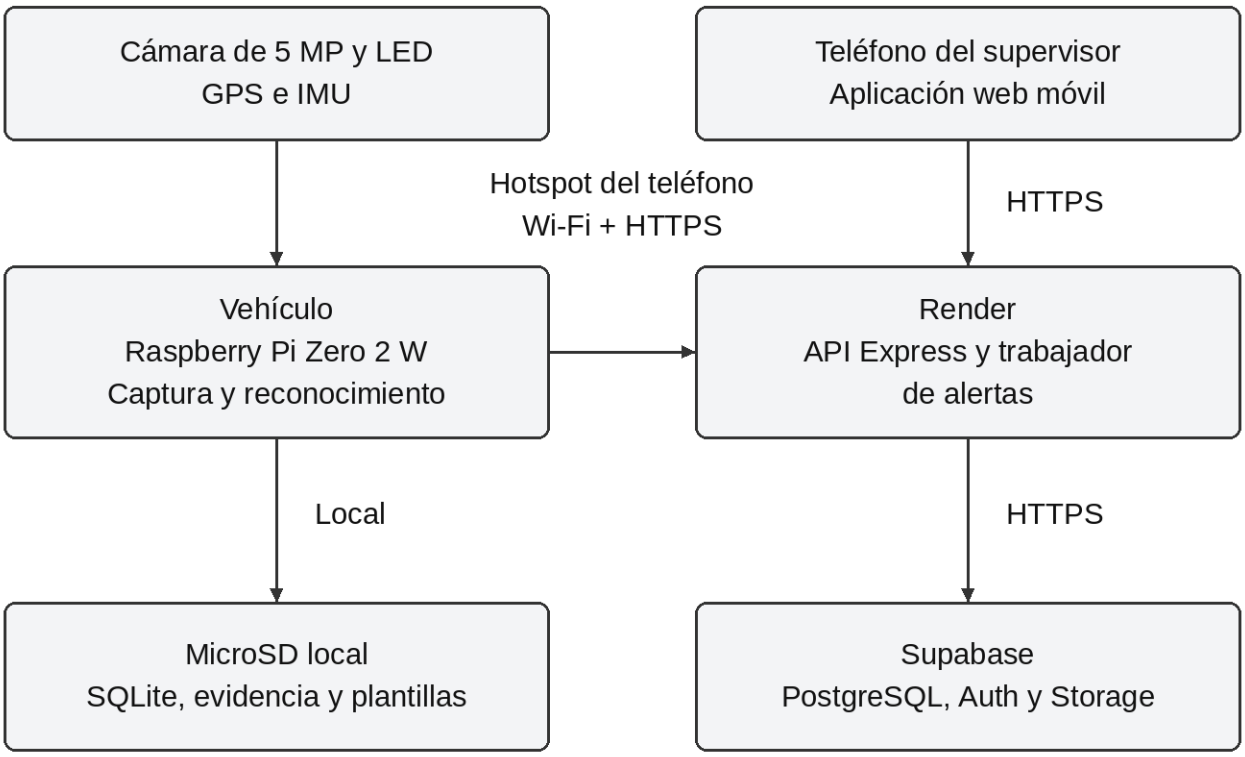


Figura 1. *Arquitectura general y despliegue de DashGuard. Las flechas indican la dirección principal del dato; cada solicitud remota tiene una respuesta de confirmación.*

En el vehículo se ejecutan los servicios de captura, identificación, recorridos, eventos y sincronización. La microSD almacena el sistema operativo, los registros SQLite, los clips y las plantillas faciales. La comparación se realiza localmente para que continúe sin Internet. No se requiere enviar rostros continuamente a la nube.

Render aloja la API Node.js/Express, el trabajador de alertas y la web estática. Supabase aloja PostgreSQL, Auth y un bucket privado de evidencia. La aplicación móvil es una aplicación web responsiva, se conecta por HTTPS y no accede a tablas o archivos privados sin autorización. La API centraliza las comprobaciones de acceso por usuario y vehículo.

La conectividad del prototipo utiliza Wi-Fi mediante el hotspot de un teléfono. Sin un enlace de Internet, una alerta se guarda localmente y se entrega después de reconectar. La aplicación consulta los avisos del vehículo asignado.

3. Arquitectura de hardware

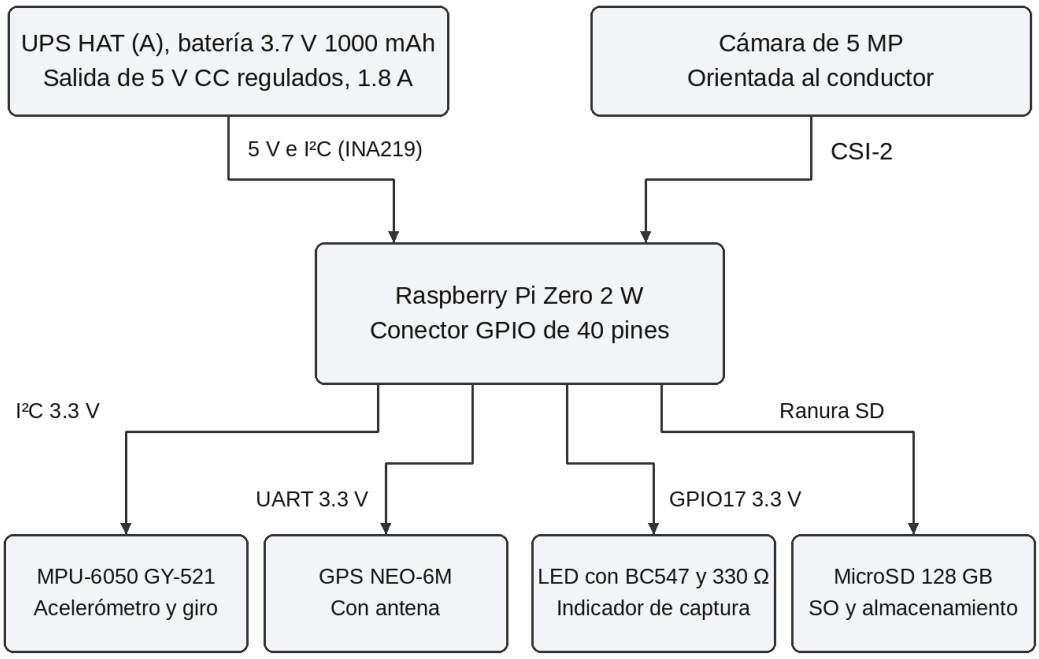


Figura 2. *Diagrama de hardware. Las líneas de sensores representan señal y referencia eléctrica compartida; el detalle de alimentación se especifica en la tabla de pines.*

La Raspberry Pi Zero 2 W dispone de 512 MB de RAM, conector CSI-2, ranura microSD y Wi-Fi de 2.4 GHz. La cámara se monta hacia el asiento del conductor y se conecta con el cable adaptador del conector CSI de la familia Zero. La alimentación es de 5 V CC regulados, con la capacidad de 2.5 A recomendada para la placa [1].

El MPU-6050 se fija al vehículo y se calibra su orientación. El NEO-6M se coloca donde tiene recepción satelital; un dato GPS inválido no se sustituye por coordenadas 0,0. El LED, conmutado por un transistor BC547, se ubica junto a la cámara, a la vista del conductor, y permanece encendido durante toda captura. El montaje permite ver al conductor sin obstruir la conducción y ninguna entrada del prototipo se conecta directamente a los 12 V del automóvil.

El UPS HAT (A) para Pi Zero incorpora una batería de polímero de litio 803040 de 3.7 V y 1000 mAh y entrega 5 V regulados con hasta 1.8 A. Se monta debajo de la Raspberry y hace contacto con los pines 1 a 6 mediante un cabezal de 2 × 3 pines, por lo que el conector GPIO de 40 pines queda libre para los sensores. La batería se recarga por el conector USB tipo C de 5 V y su monitor INA219 informa tensión, corriente y carga por el bus I²C. El respaldo sostiene el cierre ordenado ante la pérdida de la alimentación externa. El inicio y el fin de la sesión de conducción se detectan con el movimiento medido por el MPU-6050, por lo que no se requiere una entrada de ignición. Ante el apagado del equipo, el recorrido activo se cierra antes de cortar la energía.

4. Asignación preliminar de pines y voltajes

La tabla usa numeración física del conector de 40 pines y, por separado, numeración BCM. Los GPIO de la Raspberry Pi trabajan con lógica de 3.3 V [2]. Ningún módulo aplica 5 V a las líneas de señal.

Conexión	Pin físico y BCM	Nivel y observación
MPU-6050 SDA	3 / GPIO2	I²C a 3.3 V; bus /dev/i2c-1.
MPU-6050 SCL	5 / GPIO3	I²C a 3.3 V; frecuencia de 100 kHz.
MPU-6050 GND y AD0	6 / GND; AD0 a GND	Referencia común; dirección 0x68.
MPU-6050 alimentación	1 / 3V3	Alimentación a 3.3 V; las resistencias pull-up de la placa GY-521 quedan referidas a 3.3 V.
GPS TX → Pi RX	10 / GPIO15	UART a 3.3 V; receptor /dev/serial0.
Pi TX → GPS RX	8 / GPIO14	Configuración del receptor; UART a 3.3 V.

Conexión	Pin físico y BCM	Nivel y observación
GPS GND	9 / GND	Unir tierra del receptor y de la Raspberry.
GPS alimentación	17 / 3V3	Alimentación a 3.3 V, dentro del intervalo de 2.7 a 3.6 V del NEO-6M [3].
LED indicador	11 / GPIO17 a la base de Q1; emisor a 14 / GND	GPIO17 activa en alto un transistor BC547 mediante 4.7 kΩ, con 100 kΩ de base a GND. El LED y su resistencia de 330 Ω se alimentan de 3V3 y conmutan por el colector.
Cámara	Conector CSI-2	Cable compatible; no se conecta a pines GPIO.
microSD	Ranura microSD	Bus de tarjeta administrado por la placa.
Alimentación Pi	2 y 4 / 5V; 6 / GND (contactos inferiores)	5 V regulados, hasta 1.8 A, desde el UPS HAT (A); la batería se recarga por USB tipo C de 5 V.
Monitor INA219 del UPS HAT	3 / GPIO2; 5 / GPIO3 (contactos inferiores)	Comparte el bus I²C con el MPU-6050; su dirección está entre 0x40 y 0x4F y no coincide con 0x68.

El MPU-6050 opera con alimentación y niveles lógicos limitados por su hoja de datos [4], por lo que el bus I²C permanece a 3.3 V. Para el UART se habilita el puerto serie y se desactiva la consola serie, de modo que el receptor GPS queda disponible en /dev/serial0. El GPIO17 del LED no interfiere con los buses I²C y UART asignados. Los contactos del UPS HAT quedan en la cara inferior de la placa, de modo que los pines 1, 3, 5 y 6 siguen disponibles por la cara superior para el MPU-6050.

5. Módulos de software

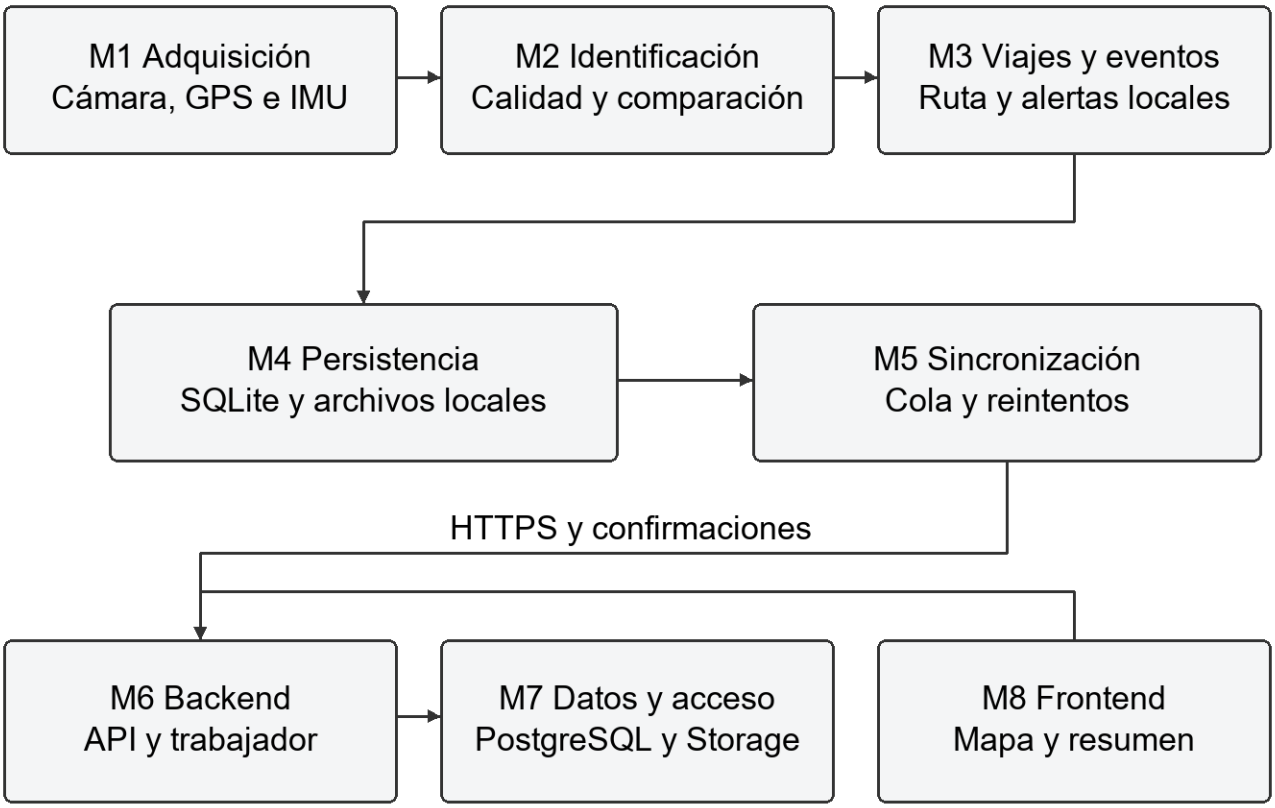


Figura 3. *Módulos de firmware, backend, datos y frontend. M1 a M5 se ejecutan en el vehículo; M6 a M8 corresponden a servicios remotos y navegador.*

El firmware se implementa como servicios Python administrados por systemd. Un único servicio controla la cámara: entrega imágenes reducidas al reconocimiento y atiende las capturas bajo demanda, de modo que ambas funciones no compiten por el sensor. Los módulos intercambian registros con identificadores estables; la captura y lectura de sensores no esperan a que termine una solicitud de Internet.

La app permite pedir una foto o video corto con GPS. El dispositivo enciende el LED durante la captura. El video corto dura diez segundos, el GPS se muestrea a 1 Hz y los umbrales de operación se almacenan en la configuración del dispositivo.

6. Responsabilidad de cada módulo

Módulo	Responsabilidad	Resultado
M1 Adquisición	Controlar cámara y LED indicador; leer GPS por UART e IMU por I²C; marcar tiempo y validez.	Imágenes, video y muestras calibradas.
M2 Identificación	Validar calidad, detectar y alinear rostro, comparar con plantillas locales vigentes. La detección usa YuNet y la comparación SFace [5].	Autorizado, no reconocido o no concluyente.
M3 Viajes y eventos	Detectar inicio y fin de sesión con el MPU-6050; abrir y cerrar el recorrido; confirmar excesos de velocidad por GPS; agrupar no coincidencias por episodio.	Recorridos, eventos y alertas locales sin duplicados.
M4 Persistencia	Guardar transacciones SQLite, cola, plantillas y archivos de captura con acceso restringido.	Registros recuperables y referencias de archivos.
M5 Sincronización	Obtener configuración y solicitudes de captura; enviar lotes y archivos; reintentar y confirmar IDs aceptados.	Cola confirmada y autorizaciones actualizadas.
M6 Backend	Autenticar, validar contratos y permisos; persistir datos; construir resumen y avisos.	API /api/v1 y alertas consultables.
M7 Datos y acceso	PostgreSQL para relaciones; Storage privado para fotos y videos; Auth para cuentas.	Integridad, sesión y acceso restringido.
M8 Frontend	Mostrar historial, mapa, resumen, resultado facial, alertas y evidencia; solicitar capturas, configurar el límite de velocidad y administrar autorizados.	Aplicación web móvil para supervisores autorizados.

7. Flujo de datos y decisiones

1. Al arrancar, systemd inicia los servicios, recupera SQLite y los pendientes y verifica cámara y sensores. El dispositivo queda en espera hasta que el MPU-6050 registra movimiento sostenido; entonces se abre un recorrido con recorrido_id único y tiempo inicial UTC. Si la hora aún no es confiable se conserva tiempo monotónico y se marca la fuente temporal.
2. Al inicio de la sesión la cámara captura al conductor con el LED encendido. M2 descarta imágenes sin rostro suficiente y compara las evaluables con plantillas locales. Una no coincidencia se confirma con tres imágenes evaluables consecutivas dentro de diez segundos. Una imagen no evaluable no identifica a un desconocido: tras dos reintentos sin imagen evaluable el resultado es no concluyente.
3. M3 registra la decisión y, ante un conductor no reconocido, crea un episodio con una alerta por destinatario e ID estable. El episodio finaliza al reconocer a un autorizado o al cerrar el recorrido. El exceso de velocidad se confirma con al menos tres lecturas GPS válidas consecutivas que cubran dos segundos por encima del límite configurado en la app.
4. M4 guarda datos y estado pendiente en la misma transacción. Las fotos y videos cortos se capturan bajo demanda y se asocian a su solicitud, fecha y ubicación GPS. La rotación de archivos mantiene al menos 2 GB de reserva para el sistema.
5. Con Internet, M5 actualiza configuración, consulta las solicitudes de captura vigentes y envía primero el recorrido y luego sus puntos, decisiones y eventos. La API confirma IDs persistidos; después se cargan archivos y se verifica integridad. Los reenvíos conservan los mismos IDs. Un archivo local no se anuncia como disponible remotamente antes de su carga completa.
6. La sesión termina cuando el MPU-6050 registra reposo sostenido o al apagar el equipo; en ambos casos se cierran el recorrido y los archivos antes de cortar energía. La web construye el resumen con datos confirmados: tiempos, duración, ruta, distancia estimada, resultado facial, alertas, velocidades válidas y eventos. Si quedan pendientes muestra un resumen parcial, ya que la ausencia de datos no equivale a cero incidentes.

8. Entradas y salidas

Componente	Entrada	Salida
Cámara / M1	Luz e imagen del conductor.	Cuadros evaluables y fotos y videos cortos.
GPS / M1	Señales satelitales y mensajes NMEA.	Posición, velocidad GPS, hora y validez.
IMU / M1–M3	Aceleración, giro y calibración.	Muestras de aceleración y giro.
LED / M1	Estado de captura activa en GPIO17.	Indicación visual al conductor durante la captura.
Identificación / M2	Rostro y plantillas autorizadas locales.	Decisión, calidad, similitud y persona_id si corresponde.
Viajes / M3	Movimiento medido por el MPU-6050, apagado y muestras.	Inicio, puntos, decisiones, eventos y cierre.
Persistencia / M4	Registros, clips y estados de envío.	Datos durables, cola y archivos protegidos.
Backend / M6	HTTPS autenticado con lotes o archivos.	Confirmación de IDs, configuración y consultas.
Alertas / M6	Episodio confirmado y destinatarios vigentes.	Aviso persistido, leído o pendiente de lectura.
Frontend / M8	Sesión, filtros, solicitudes del supervisor y respuesta de API.	Mapa, resumen, alertas y evidencia autorizada.
Propietario	Formularios de personas y permisos.	Alta, revocación y destinatarios de avisos.

9. Protocolos de comunicación

Enlace	Protocolo y parámetros	Uso
Cámara ↔ Raspberry	CSI-2 y Picamera2.	Captura y administración de imágenes/video.
IMU ↔ Raspberry	I²C, 100 kHz, 0x68 con AD0 a GND.	Lectura de acelerómetro y giroscopio.
GPS ↔ Raspberry	UART 9600 bit/s, 8N1, sentencias NMEA a 1 Hz.	Leer posición, velocidad, hora y calidad de la señal.
LED ↔ Raspberry	Salida digital GPIO a 3.3 V sobre transistor BC547.	Indicar captura activa.
UPS HAT ↔ Raspberry	I²C, monitor INA219.	Leer tensión, corriente y carga de la batería.
Servicios locales	Colas internas y SQLite 3, modo WAL.	Separar adquisición de persistencia y envío.
Dispositivo ↔ API	Wi-Fi 2.4 GHz + HTTPS TLS ≥1.2; JSON UTF-8.	Lotes, configuración, autorizaciones y estado.
API ↔ Supabase	HTTPS con SDK y credencial de servidor.	Persistencia de tablas y archivos privados.
Navegador ↔ API	HTTPS y token de usuario.	Historial, resumen, avisos y recursos.
Archivo de evidencia	Carga multipart con ID y hash SHA-256.	Transferencia independiente del lote JSON.

Se selecciona REST sobre HTTPS; no se necesita MQTT en esta fase. El dispositivo consulta las solicitudes de captura cada 5 s mientras tiene conexión. Las alertas se guardan en la base de datos y el navegador las consulta cada 15 s mientras está activo. La respuesta del servidor confirma la persistencia de la alerta; su lectura por el supervisor se registra por separado. El mapa utiliza Leaflet 1.9 y teselas de OpenStreetMap conforme a la atribución y la política de uso del proveedor.

10. Formato preliminar de mensajes

Los mensajes usan schema_version, IDs estables, tiempos ISO 8601 UTC y unidades explícitas. El dispositivo conserva tiempo de captura; el backend agrega recibido_utc. Coordenadas y velocidad inválidas se envían como null con sus indicadores de validez. El token viaja en Authorization, nunca en el JSON.

Ejemplo de lote de posición GPS para POST /api/v1/telemetria/lotes:


```
{
  "schema_version": 1,
  "lote_id": "lote-001",
  "dispositivo_id": "dashguard-001",
  "recorrido_id": "recorrido-001",
  "puntos": [
    {
      "punto_id": "punto-001",
      "secuencia": 1,
      "capturado_utc": "2026-10-21T18:00:00Z",
      "gps_valido": true,
      "latitud": 32.46088,
      "longitud": -116.861958,
      "velocidad_valida": true,
      "velocidad_kmh": 28.4
    }
  ]
}
```


Los identificadores se presentan abreviados para facilitar la lectura; la implementación genera UUID. El alta de recorrido se envía antes de sus puntos. La respuesta del lote devuelve aceptados, rechazados y motivo por ID; el dispositivo solo retira de la cola los confirmados.

10.1 Mensaje de evento y disponibilidad

Ejemplo de episodio facial para POST /api/v1/eventos:


```
{
  "schema_version": 1,
  "evento_id": "evento-001",
  "dispositivo_id": "dashguard-001",
  "recorrido_id": "recorrido-001",
  "capturado_utc": "2026-10-21T18:00:04Z",
  "tipo": "CONDUCTOR_NO_RECONOCIDO",
  "resultado": "NO_RECONOCIDO",
  "persona_id": null,
  "calidad": "EVALUABLE",
  "config_revision": 3,
  "evidencia_id": "video-001"
}
```


El tipo EXCESO_VELOCIDAD registra vehículo, velocidad, límite aplicado, fecha, hora y posición GPS, y se confirma con al menos tres lecturas válidas consecutivas que cubran dos segundos sobre el límite. La similitud facial y modelo_version se guardan con acceso restringido para la validación y no se interpretan como un porcentaje de certeza.

Cada alerta conserva alerta_id, evento_id, destinatario_id, detectada_utc, recibida_utc y leida_utc opcional. Cada solicitud de captura conserva solicitud_id, tipo, solicitante, vence_utc y estado PENDIENTE, EN_EJECUCION, COMPLETADA, FALLIDA o VENCIDA. Cada evidencia tiene evidencia_id, tipo FOTO o VIDEO_CORTO, capturado_utc, sha256 y estado PENDIENTE, DISPONIBLE o NO_DISPONIBLE; solo se marca DISPONIBLE tras verificar archivo, hash y referencia persistida.

Las operaciones POST /mediciones y GET /mediciones de la prueba de concepto se sustituyen por las operaciones versionadas /api/v1 de la sección siguiente.

11. Endpoints y operaciones principales

Operación	Autenticación	Propósito
POST /api/v1/recorridos	Dispositivo	Crear el recorrido de forma idempotente.
PATCH /api/v1/recorridos/{id}/cierre	Dispositivo	Cerrar o marcar interrumpido su propio recorrido.
POST /api/v1/telemetria/lotes	Dispositivo	Persistir puntos y mediciones por IDs.
POST /api/v1/eventos	Dispositivo	Registrar eventos y derivar alertas.
GET /api/v1/dispositivo/config	Dispositivo	Obtener límite de velocidad, versión de autorizaciones y parámetros.
POST /api/v1/evidencias	Dispositivo	Cargar foto o video con metadatos y hash.
GET /api/v1/recorridos	Usuario	Filtrar por vehículo y fecha permitidos.
GET /api/v1/recorridos/{id}/resumen	Usuario	Consultar ruta, identidad, alertas y eventos.
GET /api/v1/evidencias/{id}/acceso	Usuario	Obtener URL firmada temporal tras verificar permiso.
POST /api/v1/vehiculos/{id}/capturas	Usuario	Solicitar foto o video corto con vencimiento.
GET /api/v1/capturas/{id}	Usuario	Consultar el estado de la solicitud de captura.
PUT /api/v1/vehiculos/{id}/limite	Usuario	Configurar el límite de velocidad y generar nueva config_revision.

Operación	Autenticación	Propósito
GET /api/v1/alertas	Usuario	Obtener avisos dirigidos al usuario.
PATCH /api/v1/alertas/{id}/lectura	Usuario	Confirmar lectura sin alterar detección.

Las respuestas de creación aceptada son 201; las operaciones repetidas responden 200 con los mismos identificadores. Un error temporal conserva el pendiente. Las respuestas 401 y 403 suspenden el envío hasta corregir identidad o permisos, y una carga incompleta nunca confirma evidencia.

11.1 Autorizaciones y registro facial

Operación	Quién la utiliza	Resultado
POST /api/v1/vehiculos/{id}/autorizados	Propietario autenticado	Crear persona y autorización; no concede acceso web automáticamente.
DELETE /api/v1/vehiculos/{id}/autorizados/{persona_id}	Propietario autenticado	Revocar autorización y generar nueva config_revision.
POST /api/v1/vehiculos/{id}/enrolamientos	Propietario autenticado	Solicitar sesión de captura supervisada junto al dispositivo.
GET /api/v1/dispositivo/operaciones	Dispositivo	Obtener solicitudes de captura, altas o revocaciones pendientes para su vehículo.
POST /api/v1/dispositivo/operaciones/{id}/resultado	Dispositivo	Confirmar ejecución de captura, registro local o eliminación de plantilla.
PUT /api/v1/vehiculos/{id}/permisos/{usuario_id}	Propietario autenticado	Conceder o revocar consulta y recepción de avisos.

El registro facial requiere al propietario y a la persona informada junto al vehículo. La web solicita la operación; el dispositivo la recibe autenticada, captura muestras y calcula la plantilla local. La captura se realiza sin conducir. El backend conserva persona_id y estado de enrolamiento, sin necesitar la plantilla para construir el resumen.

La revocación remota se aplica en el dispositivo al recibir la nueva configuración; sin Internet se usa la última revisión almacenada y se señala su antigüedad. Al reconectar se actualizan autorizaciones antes de nuevos envíos y se elimina la plantilla revocada. En un equipo desconectado la revocación surte efecto al recuperar la conexión.

12. Modelo de autenticación y protección

12.1 Usuarios y dispositivos

Los usuarios inician sesión con Supabase Auth. La web envía un token de acceso de corta duración a la API. El backend valida firma, emisor, audiencia y expiración, y consulta el permiso usuario–vehículo antes de entregar datos. El propietario administra autorizados y familiares; el supervisor consulta solo los vehículos asignados [6].

Cada dispositivo recibe durante aprovisionamiento un token aleatorio propio, vinculado a dispositivo_id y vehículo. Se envía en Authorization: Bearer por HTTPS. El backend conserva un hash del token, permite rotarlo o revocarlo y evita que un dispositivo escriba datos de otro. El reconocimiento facial no reemplaza el inicio de sesión de la aplicación.

12.2 Credenciales y variables de entorno

En Render se configuran SUPABASE_URL, SUPABASE_SERVER_KEY y los parámetros de validación de Auth como secretos del servicio. DATABASE_URL solo se agrega si se utiliza conexión SQL directa. No se incluyen claves privadas en el frontend, repositorio, imágenes, ejemplos JSON ni registros. El repositorio contiene .env.example con nombres y valores ficticios; .env se excluye mediante .gitignore.

En la Raspberry, el token de dispositivo se guarda fuera del repositorio en un archivo de credenciales restringido al servicio, con permisos mínimos. Las plantillas faciales y SQLite usan acceso limitado; los logs omiten rostros, tokens y contraseñas. La microSD no se cifra en el prototipo: la protección local se basa en permisos del sistema y el cifrado en reposo se aplica en el almacenamiento de Supabase.

El bucket de evidencia es privado. La API verifica el permiso y genera enlaces firmados de duración corta; la clave de servidor permanece solo en backend [7]. Se informa al conductor sobre finalidad y datos capturados, se permite la revocación y eliminación de sus plantillas y la retención de evidencia sigue la política de la especificación de requisitos.

13. Operación sin Internet y recuperación

Condición	Comportamiento esperado
Sin Internet	Continuar verificación facial al inicio de sesión y registro GPS e IMU. Las solicitudes remotas requieren conexión. Guardar recorridos, decisiones, eventos y alertas en SQLite; conservar clips en microSD.
Red recuperada	Actualizar configuración; enviar recorrido, lotes y eventos con los mismos IDs; confirmar registros antes de liberar pendientes. Subir evidencia por separado con hash.
Reinicio del servicio	Recuperar el mismo recorrido activo dentro de la sesión de conducción. systemd reinicia el servicio sin crear recorridos duplicados.
Apagado normal	Guardar tiempo final, cerrar transacciones y archivos y persistir la cola sin esperar a Internet. El siguiente inicio de sesión crea otro recorrido.
Corte inesperado	Al arrancar, recuperar registros confirmados y marcar el recorrido anterior INTERRUMPIDO con fin desconocido. Aislar el archivo abierto y verificarlo antes de ofrecerlo.
Almacenamiento lleno	Aplicar la política de retención de capturas y respetar la reserva de espacio. Sin espacio escribible se rechaza la captura y se informa el error.
Cámara o sensor con falla	Indicar la función no disponible y mantener las independientes. Una cámara fallida produce resultado no concluyente, no una alerta de conductor no reconocido.

La outbox SQLite conserva entidad_id, tipo, payload, intentos, siguiente_reintento y estado. El backend aplica unicidad por ID y relaciones válidas. Los reintentos usan espera creciente con límite, no bloquean adquisición y diferencian errores de red de errores de autorización. Tras un corte de energía se recuperan los archivos cerrados; el video que estaba en curso puede perderse.

14. Modelo general de datos

Entidad o tabla	Relaciones y contenido
usuarios / permisos_vehiculo	usuario_id de Auth; vehículo, rol, permiso de consulta y de avisos. Conducir autorizado es una relación distinta.
vehiculos / dispositivos	Un vehículo vincula sus recorridos, autorizaciones y límite de velocidad. Cada dispositivo tiene ID, credencial hash, estado y config_revision.
personas_autorizadas	persona_id, vehículo, rol conductor, vigencia, estado de enrolamiento. La plantilla permanece local.
recorridos	recorrido_id, dispositivo, inicio, fin opcional, estado ACTIVO/FINALIZADO/INTERRUMPIDO y calidad temporal.
puntos_gps / mediciones	Pertenecen a recorrido; secuencia, tiempo, validez, posición, velocidad km/h y valores IMU cuando corresponda.
decisiones_identificacion	Recorrido, instante, resultado, persona_id opcional y versión del criterio. NO_CONCLUYENTE se distingue de NO_RECONOCIDO.
eventos / alertas	Evento vinculado al recorrido y evidencia; alerta vinculada a evento y destinatario. Restricción única por episodio y destinatario.
evidencias / solicitudes / operaciones	Referencia privada, tipo, hash y disponibilidad. Solicitudes de captura con estado y vencimiento. Operaciones de enrolamiento y revocación con ID y confirmación.

SQLite almacena el equivalente local necesario para recuperación y sincronización. PostgreSQL conserva los datos consultables remotos; Storage contiene el archivo binario. El backend arma el resumen por relaciones e intervalos temporales. La velocidad procede del GPS; no se obtiene convirtiendo directamente la aceleración a km/h.

15. Tecnologías y versiones principales

La tabla presenta las tecnologías seleccionadas para la implementación. Las versiones exactas de las bibliotecas secundarias se registran en los archivos de dependencias de cada módulo.

Capa	Selección principal	Uso
Embebido	Raspberry Pi Zero 2 W; Raspberry Pi OS Lite Legacy, Debian 12, 64 bits	Base sin escritorio; edición Bookworm disponible con actualizaciones de seguridad [8].
Firmware	Python 3.11; systemd; SQLite 3	Servicios, transacciones locales y recuperación.
Cámara e identidad	Picamera2; OpenCV 4.10 o superior; YuNet y SFace ONNX	Captura, detección y comparación facial en ARM64. La versión y el hash de los modelos se registran en cada despliegue.
Backend	Node.js 24 LTS; Express 5.x	API y trabajador de alertas sobre la línea LTS vigente [9, 10].
Persistencia remota	Supabase PostgreSQL 17	Tablas relacionales con unicidad por ID.
Identidad y archivos	Supabase Auth y Storage, servicios administrados	Cuentas de usuario y bucket privado de evidencia.
Frontend	HTML5, CSS3, JavaScript ES2022; Leaflet 1.9	Interfaz móvil; mapa, historial, capturas y alertas.
Transporte y video	REST /api/v1; JSON schema_version 1; TLS ≥1.2; H.264/MP4	Contrato estable y reproducción de archivos confirmados.

El principal riesgo de la arquitectura es el desempeño del reconocimiento facial en la Raspberry Pi Zero 2 W. Para acotarlo, la verificación se ejecuta solo al inicio de la sesión, sobre imágenes reducidas y sin grabación de video simultánea. La precisión se reporta con las muestras propias del proyecto y todo cambio de plataforma se justifica en la bitácora.

16. Matriz de requisitos y componentes

La matriz usa los identificadores de la especificación de requisitos del sistema. La columna de comprobación resume el criterio de aceptación de cada requisito.

Requisito	Componente responsable	Comprobación prevista
RF-01 Captura remota de evidencia	M8, M6, M5, M1, cámara de 5 MP y microSD	Cinco fotos y cinco videos de diez segundos íntegros y asociados a su solicitud.
RF-02 Configuración del límite	M8, M6, PostgreSQL y M5	Cinco cambios aplicados en el vehículo correcto; valores inválidos rechazados.
RF-03 Inicio y cierre de sesión	MPU-6050, M3, M4 y systemd	Cinco sesiones con recorrido único y cierre persistido; sin inicio falso en reposo.
RF-04 Registro de ubicación	GPS NEO-6M, M1 y M4	≥570 de 600 puntos previstos con señal válida.
RF-05 Registro y verificación facial	Cámara, M2, M5, M6 y M8	≥17/20 autorizados reconocidos y ≥18/20 no registrados rechazados; diez no concluyentes sin alerta.
RF-06 Exceso de velocidad	GPS, M3, M6 y M8	Diez secuencias confirmadas y cero eventos en diez interrumpidas.
RF-07 Indicador visual de captura	LED en GPIO17 y M1	LED activo en diez capturas y tres fallas; apagado en ≤1 s.
RF-08 Conexión por hotspot	Wi-Fi de la Raspberry y M5	Conexión y reconexión en ≤60 s.
RF-09 Sincronización de datos	M5, M6 y M7	100 puntos y diez eventos en ≤120 s; reenvíos sin duplicados.

Requisito	Componente responsable	Comprobación prevista
RF-10 Autenticación y validación de API	M6 y Supabase Auth	Solicitudes válidas aceptadas; inválidas y sin permiso rechazadas con 4xx.
RF-11 Persistencia en la nube	M6, PostgreSQL y Storage	Registros conservados después de reiniciar el backend.

16.1 Requisitos funcionales de consulta y operación

Requisito	Componente responsable	Comprobación prevista
RF-12 Historial de recorridos y alertas	M6, PostgreSQL y M8	Cinco recorridos y diez excesos filtrados por fecha y vehículo.
RF-13 Trayectoria en mapa	M6 y M8 con Leaflet	100 puntos en secuencia; huecos sin señal indicados.
RF-14 Carga y asociación de evidencia	M4, M5, M6 y Storage	Archivos íntegros con hash igual al origen.
RF-15 Consulta de evidencia	M6, Storage y M8	Fotos y videos visibles; cinco estados de solicitud mostrados.
RF-16 Distancia estimada	M6, PostgreSQL y M8	Diferencia ≤1 % frente al cálculo independiente.
RF-17 Operación sin Internet	M1–M5, SQLite y microSD	30 minutos sin Internet y sincronización sin duplicados.

El umbral facial se fija con muestras separadas de las de registro. Se reportan por separado el rechazo incorrecto de autorizados, la aceptación incorrecta de no registrados y la proporción de resultados no concluyentes. Un resultado no reconocido es una estimación y no la confirmación de un uso indebido. La detección de suplantación con fotografías o video queda fuera del alcance del prototipo.

16.2 Requisitos no funcionales

Requisito	Componente responsable	Comprobación prevista
RNF-01 Almacenamiento	M4 y microSD de 128 GB	Reserva mínima de 2 GB; capturas rechazadas sin espacio.
RNF-02 Latencia de captura remota	M8, M6, M5 y M1	Foto en ≤30 s y video de diez segundos en ≤60 s.
RNF-03 Formato de evidencia	Cámara de 5 MP y M1	JPEG y MP4 H.264 de 1280 × 720; video a ≥15 cuadros/s.
RNF-04 Tiempo de arranque	systemd y M1–M4	Servicios listos en ≤90 s.
RNF-05 Sincronía temporal	M1, M3 y M6	Diferencia ≤2 s con fuente temporal válida.
RNF-06 Tiempo de consulta	M6, PostgreSQL y M8	100 recorridos y 1000 puntos en ≤5 s.
RNF-07 Entrega de alertas	M3, M5, M6 y M8	Alerta visible en ≤30 s desde la confirmación local.
RNF-08 Integridad	M4–M7	Datos e IDs conservados; hashes SHA-256 iguales.
RNF-09 Privacidad y almacenamiento	M4, M6 y Supabase	Accesos sin permiso rechazados; almacenamiento privado con cifrado del proveedor.
RNF-10 Cifrado de comunicación	TLS en M5, M6 y M8	HTTPS con TLS ≥1.2 y rechazo de certificado inválido.
RNF-11 Protección de credenciales	Supabase Auth, Render y M6	Sin secretos en frontend, repositorio o registros.
RNF-12 Compatibilidad móvil	M8	Funciones completas en dos teléfonos; ancho de 390 px.
RNF-13 Recuperación tras reinicio	systemd, SQLite y M4	Cinco reinicios con datos y archivos cerrados recuperados.
RNF-14 Usabilidad	M8	Cuatro de cinco participantes completan las tareas en ≤5 min.
RNF-15 Autonomía energética	UPS HAT Xicoole y batería	Una hora con GPS, Wi-Fi y 12 fotos sin reinicio.

17. Orden de implementación y validación

La implementación inicia con la cámara dirigida al conductor y un ensayo local de identificación con personas autorizadas y no registradas. Se miden latencia, RAM, temperatura y errores mientras operan GPS e IMU, para confirmar el desempeño de la plataforma.

Después se integran la persistencia SQLite, la detección de sesión, la captura bajo demanda y la sincronización idempotente. El backend existente se amplía con las operaciones versionadas, los permisos por vehículo y las alertas persistidas. Finalmente se completa la aplicación web y se ejecutan las pruebas sin Internet, con reinicios y con cargas interrumpidas.

El cierre técnico registra para cada requisito su resultado, condiciones, fecha y evidencia. Los cambios posteriores a esta arquitectura se justifican en la bitácora del proyecto.

18. Referencias técnicas

[1] Raspberry Pi Ltd. Raspberry Pi Zero 2 W Product Brief. https://datasheets.raspberrypi.com/rpizero2/raspberry-pi-zero-2-w-product-brief.pdf

[2] Raspberry Pi. Documentación de hardware y configuración GPIO/UART. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

[3] u-blox. NEO-6 Data Sheet, GPS.G6-HW-09005. https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_(GPS.G6-HW-09005).pdf

[4] InvenSense. MPU-6000 and MPU-6050 Product Specification, revisión 3.4. https://www.cdiweb.com/datasheets/invensense/mpu-6050_datasheet_v3%204.pdf

[5] OpenCV. DNN based Face Detection And Recognition. https://docs.opencv.org/4.x/d0/dd4/tutorial_dnn_face.html

[6] Supabase. Auth. https://supabase.com/docs/guides/auth

[7] Supabase. Storage Access Control. https://supabase.com/docs/guides/storage/security/access-control

[8] Raspberry Pi. Raspberry Pi OS downloads, edición Legacy Lite. https://www.raspberrypi.com/software/operating-systems/

[9] Node.js. Release Working Group y calendario LTS. https://github.com/nodejs/Release

[10] Express. API Reference 5.x. https://expressjs.com/en/5x/api/

Consulta de documentación técnica: 21 de octubre de 2026.